Jawaban Soal Teori - Sistem Informasi

Nama: Muhamad Alvin Ramdhan | **NPM:** 20241320035

1. Perbedaan antara aturan bisnis dan logika teknis dalam Sistem Informasi

Aturan Bisnis (Business Rules) adalah kebijakan, regulasi, atau prosedur operasional yang mendefinisikan bagaimana sebuah organisasi menjalankan bisnisnya. Ini berasal dari kebutuhan manajemen.
Contoh (SI Mahasiswa): (a) Mahasiswa hanya dapat mengambil maksimal 24 SKS jika IPK lebih dari 3.0. (b) Mahasiswa dinyatakan DO jika masa studi melebihi 14 semester.

Logika Teknis (Technical Logic) adalah cara sistem komputasi diimplementasikan di belakang layar untuk memenuhi aturan bisnis tersebut. Ini mencakup arsitektur, algoritma, dan manajemen basis data.
Contoh (SI Mahasiswa): (a) Password akun mahasiswa dienkripsi menggunakan algoritma *Bcrypt* sebelum disimpan di database. (b) Data presensi disimpan sementara di *Redis cache* sebelum ditulis ke tabel MySQL.

2. Mengapa NIM mahasiswa disimpan sebagai string (str) dan bukan integer (int)?

NIM (Nomor Induk Mahasiswa) sebaiknya disimpan sebagai `string` karena NIM berfungsi sebagai kode identitas unik, bukan nilai numerik yang akan dihitung (ditambah, dikurang, dll).
Konsekuensi jika disimpan sebagai integer:

- **Hilangnya angka nol di depan (Leading Zeros):** Jika NIM adalah `002413`, tipe integer akan menyimpannya sebagai `2413`, merusak format data yang sebenarnya.
- **Keterbatasan karakter:** Jika suatu saat format NIM berubah dan membutuhkan huruf (misalnya `2024-TI-001`), tipe integer akan menyebabkan error (Type Error).
- **Batasan memori:** Pada beberapa bahasa pemrograman atau database, tipe data integer memiliki batas maksimum yang mungkin terlampaui oleh NIM yang sangat panjang.

3. Perbedaan operator logika "and" dan "or" beserta contohnya

Operator `and`: Evaluasi bernilai *True* (benar) hanya jika **semua** kondisi yang diuji bernilai benar. Jika salah satu saja salah, maka hasilnya salah.
Contoh Aturan: Syarat sidang skripsi adalah `(SKS Lulus >= 144) and (Status Keuangan == "Lunas")`. Kedua syarat wajib dipenuhi.

Operator `or`: Evaluasi bernilai *True* jika **salah satu atau lebih** kondisi yang diuji bernilai benar. Hasilnya salah hanya jika semua kondisi salah.
Contoh Aturan: Mahasiswa dibebaskan dari denda keterlambatan buku perpustakaan jika `(Alasan == "Sakit dengan Surat Dokter") or (Alasan == "Cuti Akademik")`.

4. Pentingnya urutan kondisi dalam if-elif-else

Urutan dalam blok `if-elif-else` sangat penting karena program mengevaluasi kondisi secara berurutan dari atas ke bawah (Top-Down). Saat sebuah kondisi terpenuhi (True), program akan mengeksekusi blok kode tersebut dan langsung keluar dari struktur seleksi, mengabaikan kondisi-kondisi di bawahnya.
Jika kondisi umum diletakkan sebelum spesifik: Kondisi spesifik tidak akan pernah dieksekusi (*unreachable code*). Misalnya, jika aturan diskon `if pembelian > 0` ditulis sebelum `elif`

5. Kapan menggunakan for loop dan while loop?

For Loop: Digunakan ketika jumlah iterasi atau perulangan sudah diketahui dengan pasti (definite iteration), seperti membaca elemen di dalam list, array, atau rentang angka.
Contoh bisnis: Menghitung total nilai IPK dari sebuah *list* yang berisi 8 nilai mata kuliah yang diambil mahasiswa di semester ini.

While Loop: Digunakan ketika jumlah iterasi tidak diketahui secara pasti dan perulangan harus terus berjalan selama sebuah kondisi masih bernilai *True* (indefinite iteration).
Contoh bisnis: Sistem pendaftaran akun akan terus menampilkan pesan "Masukkan password yang lebih kuat" (loop) selama panjang input password dari user kurang dari 8 karakter.

6. Fungsi klausa "else" pada for loop dan while loop di Python

Di Python, klausa `else` pada loop (baik `for` maupun `while`) berfungsi untuk mengeksekusi sebuah blok kode **hanya jika perulangan selesai secara normal** sampai akhir, tanpa dihentikan secara paksa oleh perintah `break`.
Perbedaannya dengan if-else: Pada `if-else`, blok `else` dieksekusi sebagai jalur alternatif jika kondisi `if` bernilai *False*. Sedangkan pada loop, `else` berarti "loop telah mencari atau memproses seluruh data dan tidak terjadi penghentian di tengah jalan" (sering digunakan dalam algoritma pencarian linear).

7. Mengapa validasi input harus dilakukan sebelum mengevaluasi aturan bisnis?

Validasi input berfungsi sebagai gerbang keamanan dan integritas data (prinsip *Garbage In, Garbage Out*). Ini memastikan bahwa data yang diproses memiliki format, tipe, dan nilai yang masuk akal sebelum logika bisnis yang kompleks diterapkan.
Risiko jika validasi diabaikan:

- **System Crash / Error:** Jika sistem meminta angka untuk umur tapi user menginput huruf, program bisa *crash* saat melakukan perhitungan matematis.
- **Cacat Logika (Logical Fallacy):** Tanpa validasi, user bisa menginput nilai "Minus" (-10) untuk jumlah SKS, yang mungkin secara tidak sengaja mem-bypass aturan bisnis (misalnya diskon biaya).
- **Kerentanan Keamanan:** Celah injeksi kode seperti SQL Injection atau XSS bisa terjadi jika karakter khusus tidak dibersihkan.

8. Konsep validasi komposit dan contohnya untuk pengajuan beasiswa

Validasi Komposit adalah proses memvalidasi data dengan menggabungkan beberapa aturan validasi tunggal (menggunakan operator logika) menjadi satu kesatuan syarat yang utuh. Sebuah data dinyatakan valid hanya jika lolos dari keseluruhan kombinasi aturan tersebut.
Contoh Implementasi Pengajuan Beasiswa:
Untuk mengajukan beasiswa, sistem tidak hanya mengecek satu hal, melainkan kombinasi syarat. Logika kompositnya bisa berupa:
`is_valid = (ipk >= 3.5) and (semester_aktif >= 3) and (pendapatan_orang_tua <= 3000000) and (status_pelanggaran == False)`
Jika salah satu saja dari kondisi tersebut gagal tervalidasi, maka variabel `is_valid` akan bernilai *False* dan pengajuan beasiswa otomatis ditolak oleh sistem.